

VET-C 模型工程實作手冊

現在請把 VET-C 模型的建置方式，寫成一份工程實作說明書。必須涵蓋所有的細節 (至少可以讓 Codex 可以去產出相關的 Code)。可以一部份一部份的寫出來。首先給我整份資料的 Volume List 以及每個 Volume 裡面要做的東西介紹。把這份資料做成一張表。然後接下來將一冊一冊的 Volume 一步一步的寫給我。

VET-C 模型工程實作說明書 Volume List

這套文件的目標是讓 Codex 可以依序產生模組、資料結構、訓練流程與測試程式。

Volume	文件名稱	主要目標	內容範圍	Codex 產出重點
Vol. 00	VET-C System Overview	定義整體系統架構	VET-C 的目的、資料流、核心模組、三層架構：Regime / Latent Field / VET Terrain	專案總架構、模組邊界、主流程圖
Vol. 01	Data Interface & Sensor Pipeline	建立輸入資料標準	Camera、ArduPilot、IMU、GPS、Battery、Motor、Control Log 的資料格式	schemas/、sensor adapter、log parser
Vol. 02	External World Encoder	建立外部世界潛空間	Camera sequence → V-JEPA Encoder → External Latent (z^{ext})	V-JEPA wrapper、video encoder、latent cache
Vol. 03	Internal State Encoder	建立無人機自身潛空間	ArduPilot 狀態 → Internal RSSM → Internal Latent (z^{int})	RSSM model、state encoder、prediction loss
Vol. 04	Projection Head	將 latent 投影成十個 Directional Nodes	($z^{\text{ext}}, z^{\text{int}}$) → R±、V±、O±、C±、B± 的初步 node embeddings	Projection Head NN、proxy grounding loss、pair loss
Vol. 05	Operational Proxy Signals	定義十個 Node 的自監督教師訊號	viability margin、value change、optionality、prediction error、policy consistency	proxy signal generator、teacher signal builder
Vol. 06	Evidence Fusion Layer	融合四種 Evidence 得到 Node Measures	Probability、Rollout Weight、Memory Weight、World Model Belief → 10 Node Measures	Evidence MLP、fusion loss、normalizer
Vol. 07	Fixed Node Topology	建立固定十節點因果拓樸	十個 nodes 的允許流向、edge mask、topology flow	topology matrix、edge strength module、topology loss
Vol. 08	Regime Reader	從 Node Measures 讀出 Regime	External Regime、Internal Regime、admissible attractor、transition token	regime codebook、lookup table、regime classifier
Vol. 09	Memory System	建立 VET-C 記憶層	Log、Episode、Pattern、Impression Tree、Memory Block	memory schema、retrieval、write rule、compression
Vol. 10	Future Cone & Rollout Engine	建立未來軌跡生成器	World Model rollout、trajectory sampling、feasible set、optionality	rollout engine、trajectory evaluator
Vol. 11	VET Terrain Functional	建立決策地形與作用量泛函	Risk / Value / Opportunity / Certainty / Belief → trajectory score	VET scorer、functional evaluator、trajectory ranking
Vol. 12	Control & Policy Interface	產生無人機控制命令	Future Cone → selected trajectory → control action	MPC / policy adapter、ArduPilot command interface
Vol. 13	Training Pipeline	定義完整訓練流程	離線預訓練、自監督 rollout、topology training、test-time training	training scripts、loss registry、checkpoints
Vol. 14	Test-Time Training & Online Adaptation	定義飛行中逐步成熟機制	只更新 Projection Head / Evidence Fusion / Memory Weight，不改安全邊界	online adapter、safe update rule、rollback
Vol. 15	Evaluation & Validation	定義測試與驗收	prediction error、node semantics、regime accuracy、control safety、mission success	unit tests、simulation tests、metrics dashboard
Vol. 16	Repository Structure & Codex Prompts	讓 Codex 產生工程專案	專案目錄、檔案命名、模組依賴、逐步 Codex Prompt	README.md、AGENTS.md、Codex prompt pack

Vol. 00 → Vol. 01 → Vol. 02 → Vol. 03 → Vol. 04 → Vol. 05 → Vol. 06

這七冊會先把最核心的鏈條建立起來:

Camera / ArduPilot

→ External / Internal Latent

→ Projection Head

→ Proxy Signals

→ Evidence Fusion

→ 10 Node Measures

Vol. 00 : VET-C System Overview

這一冊直接定位成 **Codex Engineering Constitution**，也就是 Codex 可以依照這份文件逐步生成整個 VET-C 平台。

整份文件中的每一冊都維持固定格式：

1. Purpose（目的）
2. Scope（範圍）
3. Theory（理論）
4. Data Structure（資料結構）
5. Interface（介面）
6. Training（訓練）
7. Runtime（執行）
8. Testing（測試）
9. Acceptance Criteria（驗收）
10. Codex Implementation Checklist（Codex 要完成的工作）

VET-C Engineering Specification Volume 00

VET-C System Overview

Version 1.0

Chapter 1. Purpose

本文件定義 VET-C (Value-Entropy-Terrain-Control) 模型之完整工程架構。

本文件不是數學論文，而是整個系統所有 Module 的最高層 Engineering Specification。

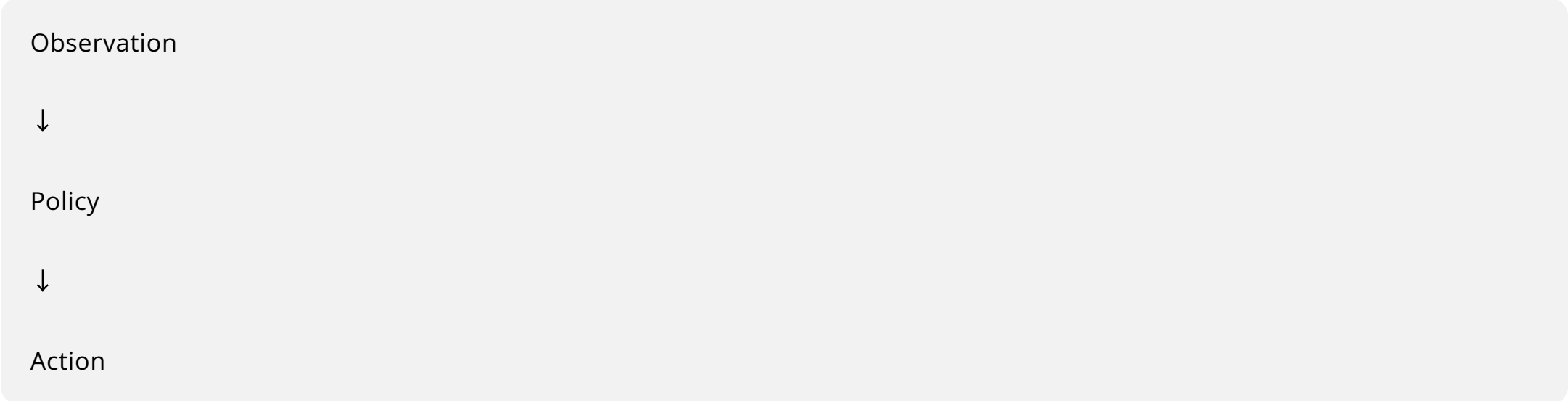
其目的包括：

- 定義所有 Module 的責任
- 定義所有資料流
- 定義所有 Latent Space
- 定義 Node Space
- 定義 Regime
- 定義 Future Cone
- 定義 Control

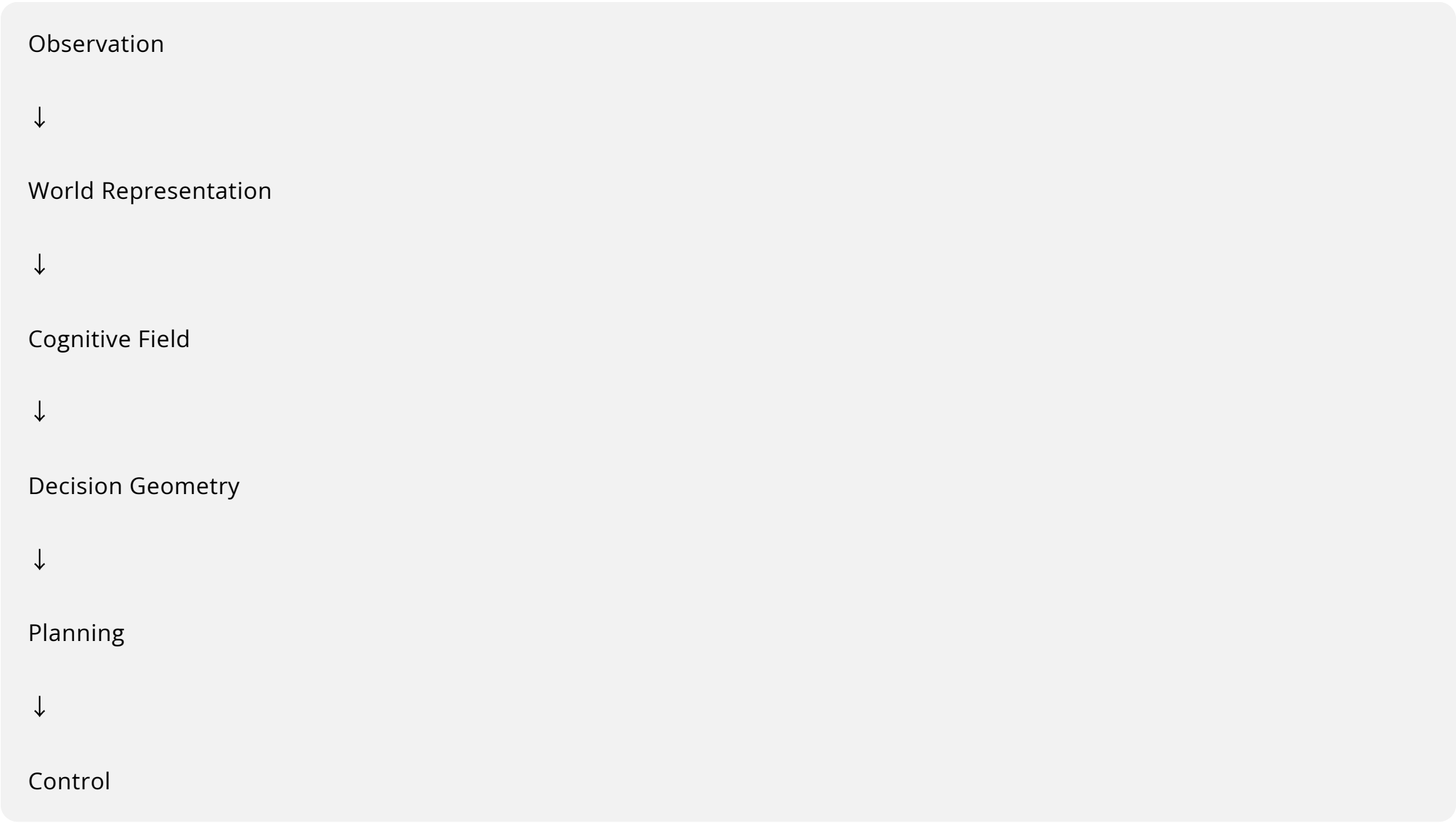
Codex 必須依照本文件建立完整 Repository。

Chapter 2. Design Philosophy

VET-C 不採用：



而是：



也就是：**世界理解（Understanding）** 先於 **控制（Control）**。

2.1 傳統 Policy-based 方法的問題

傳統強化學習或端對端控制系統的核心假設是：給定足夠的訓練資料，模型可以直接從 Observation 映射到 Action，而不需要顯式地理解世界。這種方法在受控環境下往往表現良好，但存在以下根本性缺陷：

- 分布外失敗（Out-of-Distribution Failure）**：當環境狀態超出訓練分布時，Policy 無法可靠地泛化，因為它從未建立對世界的因果結構理解。
- 不可解釋性（Lack of Interpretability）**：決策是黑箱映射，無法解釋「為何做出此決策」，亦無法提供安全邊界的形式化保證。
- 無法分離規劃與執行**：Policy 將高層規劃與低層控制混為一談，使得系統難以模組化、驗證與調試。

2.2 VET-C 的核心主張：先理解，再控制

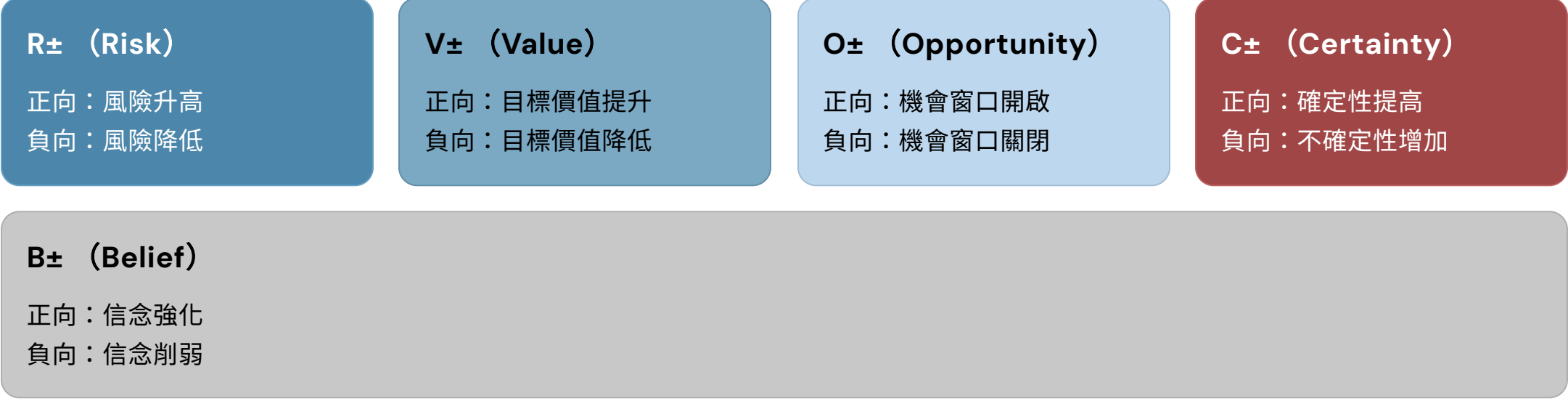
VET-C 的根本主張是：**有意義的決策必須建立在對世界的內部表徵之上**。系統在採取任何行動之前，必須先構建兩個互補的潛空間（Latent Space）：

- External Latent Space**：對外部環境的壓縮表徵，捕捉地形、障礙物、氣流、威脅等外部狀態的本質結構。
- Internal Latent Space**：對無人機自身狀態的壓縮表徵，包含速度、姿態、能源、感測器健康度等內部狀態。

這兩個潛空間共同構成 World Representation，是後續所有推理的基礎。系統不直接從 Observation 決策，而是先將感知資料投影至這個結構化的理解空間，再從中推導行動。

2.3 Cognitive Field（認知場）的角色

World Representation 是高維度的潛空間，本身並不直接可解釋。Cognitive Field 的職責是將潛空間投影至十個具有語意的認知維度，稱為 **Directional Nodes**：



這十個節點構成 **Node Space**，是系統進行所有高層推理的認知座標系。每個節點的數值代表系統在該維度上的當前認知強度，使決策過程從黑箱映射轉變為可觀測、可解釋的幾何運算。

2.4 Decision Geometry（決策幾何）

在 Node Space 中，VET-C 定義了兩個核心概念，將決策問題幾何化：

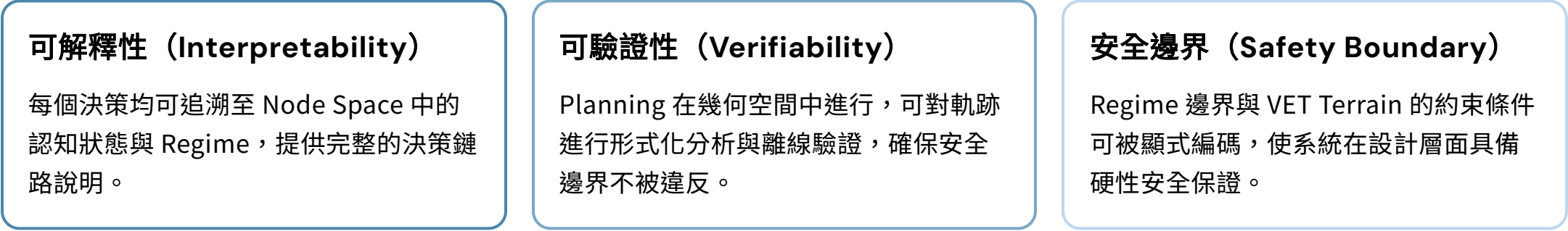
- Regime（狀態域）**：Regime 是 Node Space 中的一個區域，代表系統當前所處的認知狀態類型。例如「高風險低確定性」或「高機會高信念」各對應不同的 Regime，系統據此選擇對應的決策策略。Regime 的邊界是可形式化定義的，使狀態轉換具有可驗證性。
- VET Terrain（決策地形）**：VET Terrain 是定義在 Node Space 上的純量場，其數值反映各認知狀態下的決策偏好。Planning 的本質是在此地形上尋找最優路徑，將決策問題轉化為幾何問題，而非依賴黑箱的端對端映射。

2.5 Planning → Control 的分離

VET-C 明確將規劃層與控制層解耦，這是系統可解釋性與安全性的關鍵設計決策：

- Planning（規劃）**：在 **Future Cone** 中展開多條候選軌跡，對每條軌跡評估其在 VET Terrain 上的累積代價、風險邊界與目標達成度，選出最優路徑。此步驟完全在認知空間中進行，不涉及實際控制指令。
- Control（控制）**：將規劃層選定的最優軌跡轉換為具體的低層控制命令（推力、姿態、速度指令等）。控制層只負責執行，不負責決策。

此分離架構帶來三項核心工程優勢：



Chapter 2 的學術論文創新性分析

2.A 學術背景：現有 World Model 方法

學術界已有大量研究採用 World Model 實現 Planning 與 Action 的分離，以下為主要代表性方法：

Dreamer / DreamerV3 (<i>Hafner et al.</i>) 在 latent space 中進行 rollout，以 actor-critic 學習 policy。然而 latent space 無語意結構 ，維度為統計壓縮碼，不具可解釋性。	MuZero (<i>Schrittwieser et al.</i>) 學習 value-equivalent model，在 search tree 中進行 planning。latent 表徵仍為 黑箱 ，無法對決策空間進行形式化分析。	TD-MPC2 (<i>Hansen et al.</i>) 結合 temporal difference 與 MPC，在 latent space 中進行 trajectory optimization。latent 無可解釋維度 ，優化目標缺乏認知語意。	RSSM-based Methods (<i>PlaNet, MBPO</i>) 以 Recurrent State Space Model 建立世界模型。latent 維度 無固定語意 ，無法對狀態轉換施加因果結構約束。
---	--	--	--

2.B VET-C 的核心差異：Semantic Latent Space

上述方法的共同特徵是：latent space 為**無約束的統計壓縮空間**，維度無固定語意，planning 在此空間中進行，本質上仍是黑箱幾何搜索。

VET-C 的根本創新在於：不使用通用 latent space，而是將 latent 投影至一個**具語意基礎的 10 維 Node Space**（R±、V±、O±、C±、B±）。這意味著：

- 每個維度具有**固定、可解釋的語意**（Risk、Value、Opportunity、Certainty、Belief）
- Latent space 不以端對端方式無約束地學習——而是由 **proxy signals 進行語意錨定 (grounding)**
- Planning 發生在一個**每個軸線均具有物理／操作語意**的空間中，而非抽象統計流形

2.C 五項具體創新優勢

① 語意潛空間 (Semantic Latent Space) 現有方法的 latent space 是統計壓縮，維度無語意。VET-C 的 Node Space 每個維度對應明確認知概念（R±、V±、O±、C±、B±），使 latent 從「壓縮碼」升級為「 認知座標 」，為可解釋 AI 提供形式化基礎。	② Regime 作為離散認知狀態 Dreamer / MuZero 的 planning 在連續 latent 中進行，無法定義「系統當前處於何種認知狀態」。VET-C 的 Regime 將 Node Space 離散化為 可命名、可驗證的狀態域 ，使 planning 策略可依 Regime 切換，具備結構化的情境感知能力。	③ Evidence Fusion 的多源信念整合 現有方法通常只有單一 world model belief。VET-C 明確融合四種 Evidence（Probability、Rollout Weight、Memory Weight、World Model Belief），使 Node Measure 反映 多源不確定性 ，更接近貝葉斯推理的精神，亦使信念來源可被追溯。	④ 固定拓樸的因果約束 (Fixed Causal Topology) 現有 latent space 方法不對維度間的因果關係做任何約束。VET-C 的 Fixed Node Topology 明確定義十個節點之間的 允許流向 ，將因果結構編碼進模型，使 planning 在因果一致的空間中進行，從根本上消除反因果推理的風險。
⑤ Test-Time Training 的安全邊界保護 現有 online adaptation 方法（如 MBPO 的 online fine-tuning）缺乏對安全邊界的形式化保護。VET-C 的 TTT 只更新 Projection Head / Evidence Fusion / Memory Weight，Regime 邊界與 VET Terrain 的安全約束 不可被在線更新覆蓋 ，實現「適應性」與「安全性」的解耦。			

2.D 學術定位總結

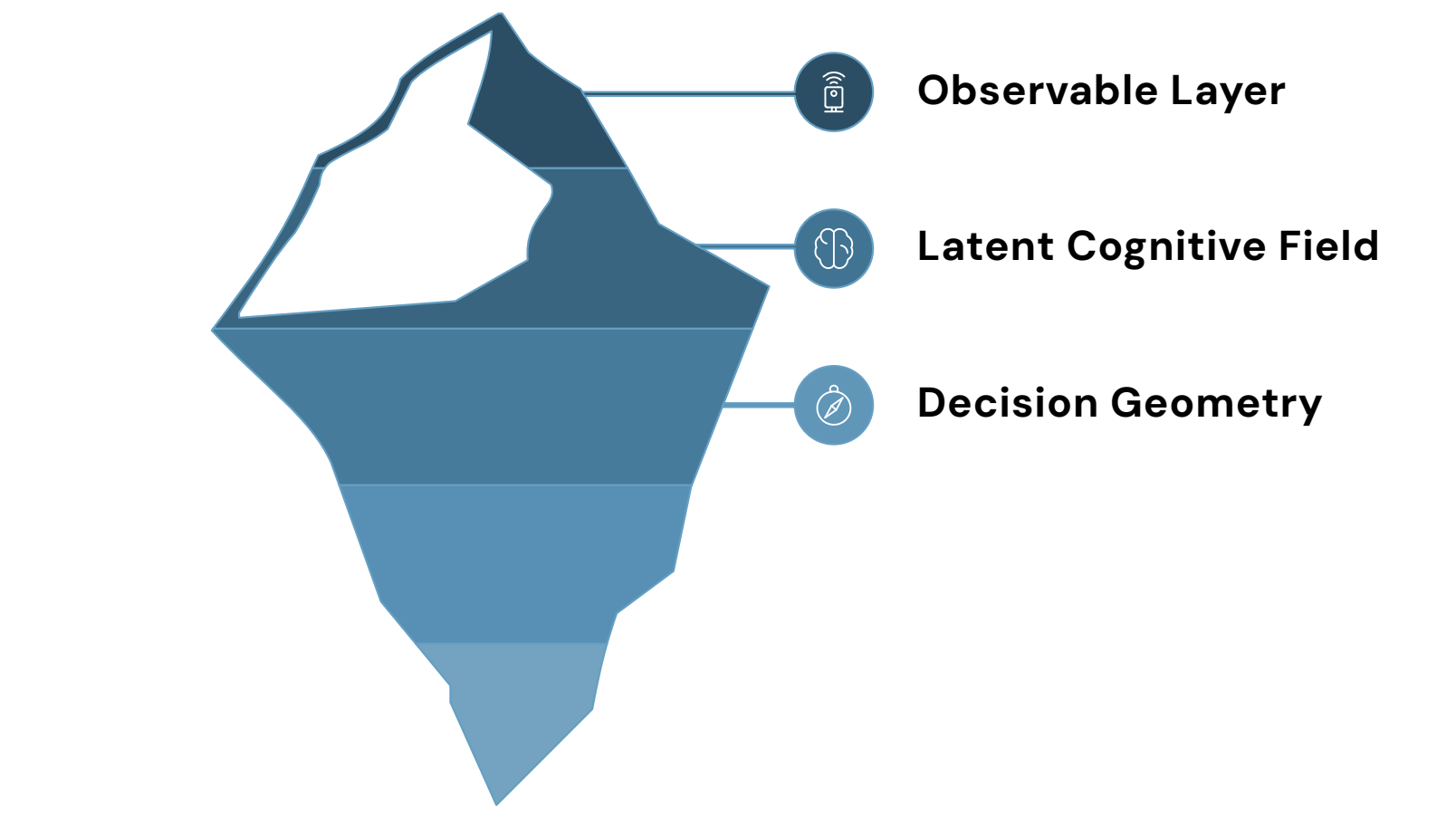
VET-C 的學術貢獻不在於提出一個預測精度更高的 world model，而在於提出一個**具有語意結構的認知中間層 (Semantic Cognitive Intermediate Layer)** ——將 world model 的 latent space 從無約束的統計壓縮空間，轉化為具有固定語意、因果拓樸與離散狀態結構的**可解釋認知幾何空間**。這使得 VET-C 在三個核心維度上系統性地超越現有 world model + planning 方法：

可解釋性 (Interpretability) 每個決策均可追溯至具語意的 Node Space 座標與 Regime 狀態，決策鏈路完整透明。	安全性 (Safety) 安全約束被形式化編碼於 Regime 邊界與 VET Terrain，在設計層面具備硬性安全保證，且不可被在線適應覆蓋。	因果一致性 (Causal Consistency) Fixed Node Topology 將因果結構顯式編碼，使 planning 在因果一致的幾何空間中進行，根本性地區別於現有黑箱 latent 方法。
--	---	--

Chapter 3. Three Cognitive Layers

整個 VET-C 可以分成三層。

這三層嚴格分離——任何一層不得繞過另一層直接存取。資料流方向嚴格向下：**Observable** → **Latent Field** → **Decision Geometry**。此分離是系統可解釋性與安全性的架構級保證，而非實作細節的選擇。



三層之間的資料流向嚴格單向，不可逆行或跨層存取。以下逐層展開說明。

Layer 1：Observable Layer

代表：可以直接量測的物理訊號。

設計原則：Observable Layer 是系統與物理世界的唯一接觸點。所有感測器資料在此層被標準化為統一格式，不做任何推理或解釋。此層的職責是「忠實記錄」，而非「理解」。

感測器來源清單：

- Camera
- IMU
- GPS
- Battery
- Motor
- Velocity
- Altitude
- Wind
- ArduPilot（飛控狀態）

感測器規格總覽

感測器來源	資料類型	輸出格式
Camera	RGB video frames	$T \times H \times W \times 3$ tensor, 30fps
IMU	加速度、角速度	6-DOF float32, 100Hz
GPS	經緯度、高度、速度	WGS84 + NED frame, 10Hz
Battery	電壓、電流、剩餘容量	float32 $\times$ 3, 10Hz
Motor	轉速、推力估計	float32 $\times$ N_motors, 50Hz
ArduPilot	飛控狀態、模式、控制輸出	MAVLink message struct, 50Hz
Wind	風速、風向估計	float32 $\times$ 3, 10Hz

輸出：Raw Observation（時間戳對齊的多模態感測器資料流）

關鍵約束：Observable Layer 不得進行任何推理。它只負責資料的採集、時間戳對齊、格式標準化與緩衝。所有解釋工作交由 Layer 2 處理。

Layer 2：Latent Cognitive Field

代表：世界模型——系統的「理解引擎」。

設計原則：Latent Cognitive Field 不回答「世界長什麼樣子」，而是回答「世界正在如何變化，以及這對無人機意味著什麼」。這一層將高維感知資料壓縮為具有語意結構的認知表徵。

世界正在如何變化？

Layer 2 的主要子元件如下：

3.2.1 External Latent（外部潛空間）

Camera sequence 經由 **V-JEPA Encoder** 壓縮為 z_{ext} ，捕捉外部環境的時序動態結構。 z_{ext} 不是影像的像素重建，而是環境變化模式的壓縮表徵——代表「外部世界正在發生什麼」的潛在編碼。

3.2.2 Internal Latent（內部潛空間）

ArduPilot 狀態序列經由 **Internal RSSM** 壓縮為 z_{int} ，捕捉無人機自身的動態狀態演化。RSSM 同時維護確定性隱狀態（ h_t ）與隨機潛狀態（ z_t ），使系統能對自身未來狀態進行預測。

3.2.3 Projection → Directional Nodes

(z_{ext}, z_{int}) 經由 **Projection Head** 投影至 10 個 Directional Nodes（**R+**, **R-**, **V+**, **V-**, **O+**, **O-**, **C+**, **C-**, **B+**, **B-**）。每個 Node 的數值由 Proxy Signal 進行語意錨定，確保投影結果具有可解釋的操作語意，而非抽象統計維度。

3.2.4 Evidence Fusion → Node Measures

四種 Evidence（**Probability**、**Rollout Weight**、**Memory Weight**、**World Model Belief**）經由 Evidence MLP 融合，得到最終的 10 個 Node Measures。Node Measures 是 Layer 2 的最終輸出，代表系統在當前時刻的完整認知狀態，反映多源不確定性的貝葉斯整合結果。

3.2.5 Regime

Regime Reader 從 Node Measures 讀出當前 **Regime**，即系統所處的離散認知狀態域。Regime 將連續的 Node Space 離散化為可命名、可驗證的狀態域，是 Layer 2 傳遞給 Layer 3 的核心語意摘要。

Layer 2 輸出：Node Measures（10 維認知座標）+ Regime（離散認知狀態）

Layer 3：Decision Geometry

代表：Decision Terrain——系統的「決策引擎」。

設計原則：Decision Geometry 不感知世界，也不理解世界——它只在 Layer 2 提供的認知座標系中，計算最優行動路徑。這一層將規劃問題轉化為幾何問題，在 Node Space 中尋找最優軌跡。

我應該往哪裡走？

Layer 3 的主要子元件如下：

3.3.1 Future Cone（未來錐）

以當前 Node Measures 為起點，World Model 向前展開 N 步，生成多條候選軌跡，構成 **Future Cone**。每條軌跡代表一種可能的未來認知狀態演化路徑，Future Cone 的寬度反映系統的不確定性程度。

3.3.2 VET Functional（決策泛函）

對 Future Cone 中的每條軌跡，VET Functional 計算其綜合評分：

- V (Value)**：目標達成度
- E (Entropy / Risk)**：風險與不確定性
- T (Terrain)**：地形可行性與安全邊界距離

最優軌跡是使 VET Functional 最小化的路徑。

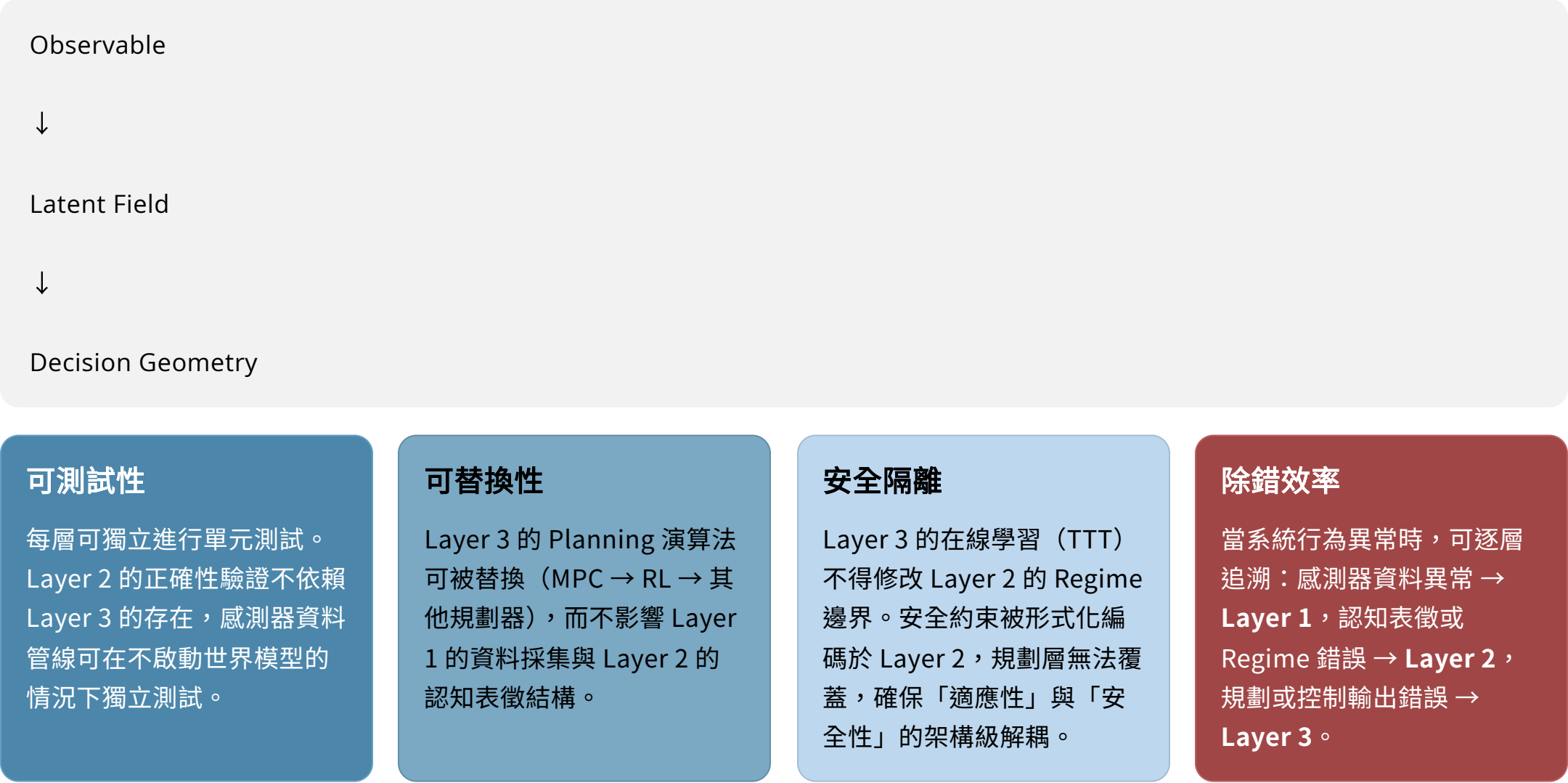
3.3.3 Planning → Control

選定最優軌跡後，**MPC / Policy Adapter** 將軌跡轉換為具體的低層控制命令，發送至 ArduPilot 執行。此步驟完成從認知空間的幾何規劃到物理空間的執行命令的轉換。

Layer 3 輸出：控制命令（發送至 ArduPilot 的 MAVLink 指令）

三層不可混淆的工程意義

三層嚴格分離不僅是設計哲學，更是工程規範的強制要求：



Chapter 3 的學術定位分析：類似研究與 VET-C 的差異

3.A 最新相關研究概覽（2024–2026）

近年學術界在 UAV World Model、Cognitive Architecture 與 Interpretable Planning 三個方向均有重要進展，以下為最具代表性的最新研究：

AirScape (2025, Tsinghua)

首個針對六自由度空中代理的生成式世界模型。以視覺輸入與運動意圖預測未來觀測序列，在 3D 空間想像能力上超越現有基礎模型 50%+。但其 latent space 仍為視覺生成導向，無語意結構，不具認知可解釋性。

ANWM — Aerial Navigation World Model (2024, Tsinghua)

以 World Model 預測未來視覺觀測，讓 UAV 依語意可行性對候選軌跡排序。引入 Future Frame Projection (FFP) 模組提供幾何先驗。但其 planning 仍依賴視覺相似度比對，非認知幾何空間中的結構化推理。

WorldVLN (2026, Tsinghua)

首個用於空中視覺語言導航的自回歸 World Action Model，將世界狀態轉換預測與動作解碼整合。引入 Action-aware GRPO 強化學習方法。但其 latent 為視覺自回歸序列，無固定語意維度，安全邊界無形式化保證。

CityNavAgent (ACL 2025)

以 LLM 為核心，設計階層式語意規劃模組 (HSPM) 與全域記憶拓樸圖，用於城市空中導航。具備語意層次分解能力，但依賴 LLM 的隱式世界知識，缺乏顯式的認知狀態表徵與因果拓樸約束。

Structured Cognitive Loop / SCL (2025)

提出 R-CCAM 模組化認知架構 (Retrieval, Cognition, Control, Action, Memory)，以 Soft Symbolic Control 作為治理層，實現零 policy 違規與完整決策可追溯性。與 VET-C 在「認知層與控制層分離」的設計哲學上最為接近，但應用於 LLM Agent，非嵌入式飛控系統。

MBCWM — Mental Bayesian Causal World Model (2025)

提出「認知一致性 (Cognitive Consistency)」作為核心學習機制，主張智能來自內部意圖世界與物理現實的語意對齊，而非像素級重建。與 VET-C 的「世界理解先於控制」哲學高度呼應，但應用於自動駕駛，且缺乏顯式的 Node Space 與 Regime 結構。

3.B VET-C 與最新研究的系統性比較

研究	Latent 語意結構	認知狀態離散化	因果拓樸約束	安全邊界形式化	在線適應安全保護	應用領域
AirScape (2025)	☒ 視覺生成	☒	☒	☒	☒	UAV 視覺生成
ANWM (2024)	☒ 視覺相似度	☒	☒	☒	☒	UAV 導航
WorldVLN (2026)	☒ 自回歸視覺	☒	☒	☒	☒	UAV VLN
CityNavAgent (2025)	△ LLM 隱式	△ 語意子目標	☒	☒	☒	城市空中導航
SCL (2025)	△ 符號約束	△ 認知階段	△ 治理層	△ 軟性約束	☒	LLM Agent
MBCWM (2025)	△ 意圖對齊	☒	△ 因果模型	☒	☒	自動駕駛
VET-C (本系統)	✓ 10維語意 Node	✓ Regime離散化	✓ Fixed Topology	✓ 形式化編碼	✓ TTT安全隔離	UAV 飛控

3.C VET-C 的三項獨特貢獻

唯一針對嵌入式飛控的語意認知架構

現有最先進的 UAV World Model (AirScape、ANWM、WorldVLN) 均以視覺生成為核心，latent space 服務於影像預測而非認知推理。VET-C 是目前已知唯一將 latent space 設計為具固定語意維度的認知座標系，並直接驅動飛控決策的系統。

Regime 離散化 + Fixed Topology 的雙重結構約束

即使是最接近 VET-C 設計哲學的 SCL 與 MBCWM，也未同時具備「離散認知狀態 (Regime)」與「固定因果拓樸 (Fixed Node Topology)」兩項結構約束。這兩項約束的組合使 VET-C 的 planning 空間具備形式化可驗證性，是現有研究中的空白。

Test-Time Training 的安全隔離機制

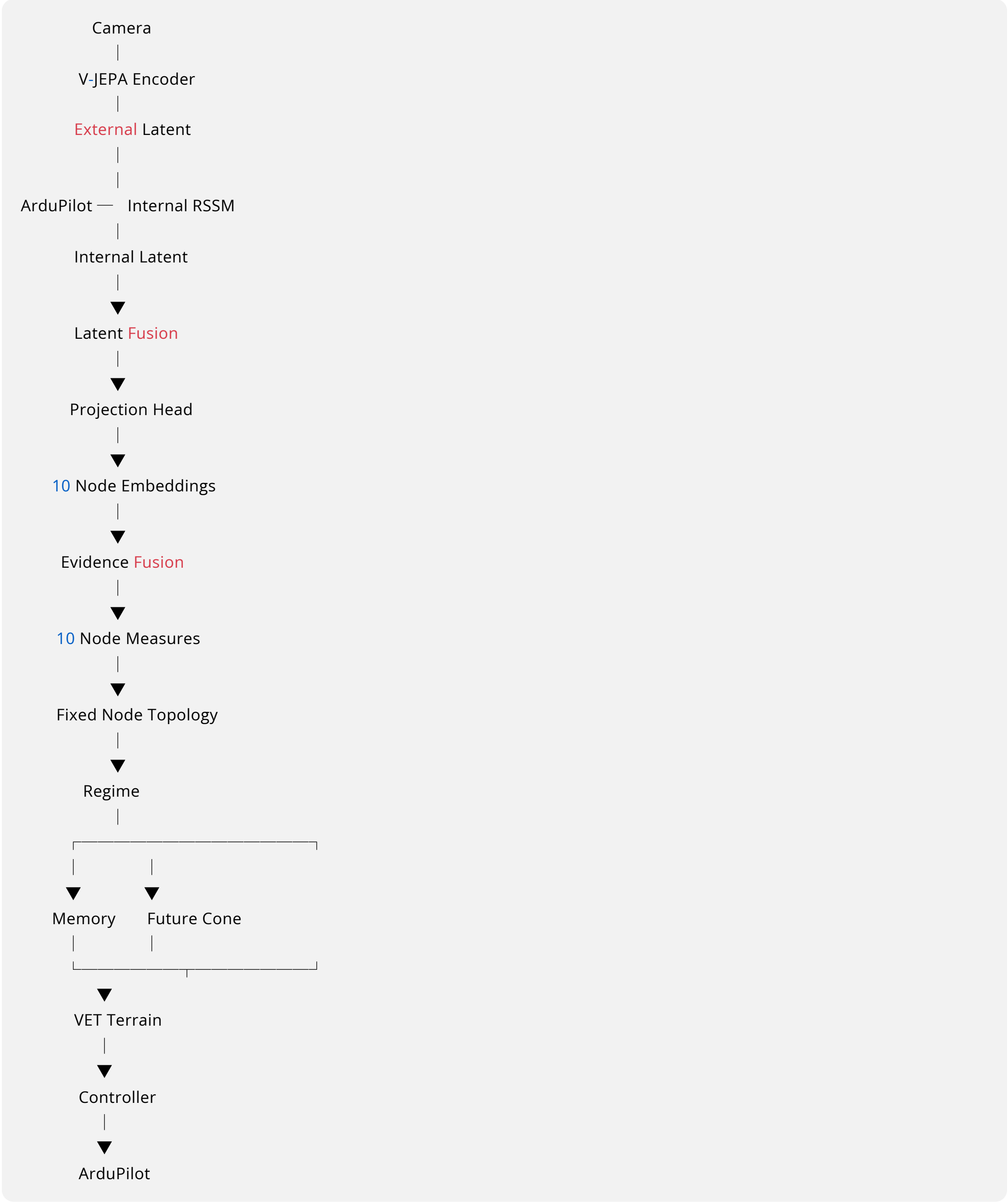
所有現有研究均未提出在線適應時的安全邊界保護機制。VET-C 的 TTT 安全隔離（只更新 Projection Head / Evidence Fusion / Memory Weight，不修改 Regime 邊界）是目前已知唯一在架構層面形式化保證「適應性不破壞安全性」的設計。

3.D 研究定位總結

VET-C 並非在現有 UAV World Model 的技術路線上做增量改進，而是開闢了一條全新的研究路徑：**以語意認知幾何空間取代視覺生成空間作為 UAV 決策的核心表徵**。這一路徑在 2024–2026 年的最新文獻中尚無直接競爭者，具備清晰的學術創新性與工程落地價值。

Chapter 4. Overall Architecture

整體資料流如下：



上圖展示 VET-C 的完整資料流，從原始感測器輸入到最終飛控命令輸出，共經過十一個主要處理階段。整條鏈路嚴格遵循三層架構（Observable → Latent Field → Decision Geometry），每個模組的輸入與輸出均有明確定義。以下逐段說明。

4.1 感知輸入層（Observable → Latent）

Camera → V-JEPA Encoder → External Latent：相機影像序列（ $T \times H \times W \times 3$ ）輸入 V-JEPA Encoder，以自監督方式學習時序動態表徵，輸出 External Latent z_{ext} 。V-JEPA 的核心優勢在於不需要像素級重建，而是在 latent space 中預測被遮蔽的未來幀，使 z_{ext} 捕捉的是環境的動態結構而非靜態外觀。

ArduPilot → Internal RSSM → Internal Latent：ArduPilot 的飛控狀態序列（速度、姿態、電池、馬達等）輸入 Internal RSSM，RSSM 同時維護確定性隱狀態 h_t 與隨機潛狀態 z_t ，輸出 Internal Latent z_{int} 。RSSM 的設計使系統能在無新觀測的情況下，純粹依靠內部動態模型預測自身未來狀態。

4.2 認知表徵層（Latent → Node Measures）

Latent Fusion： z_{ext} 與 z_{int} 在此進行融合，得到統一的聯合潛空間表徵（ z_{ext}, z_{int} ）。融合方式可為 concatenation 或 cross-attention，具體由 Vol. 04 定義。

Projection Head → 10 Node Embeddings：聯合潛空間經由 Projection Head（多層 MLP）投影至 10 個 Directional Node Embeddings（R+, R-, V+, V-, O+, O-, C+, C-, B+, B-）。每個 Node Embedding 的語意由 Proxy Signal 進行錨定訓練，確保投影結果具有可解釋的操作語意。

Evidence Fusion → 10 Node Measures：四種 Evidence（Probability、Rollout Weight、Memory Weight、World Model Belief）與 Node Embeddings 一同輸入 Evidence MLP，融合後得到最終的 10 個 Node Measures。Node Measures 是系統當前認知狀態的完整數值表徵。

Fixed Node Topology：Node Measures 通過 Fixed Node Topology 模組，依照預定義的因果流向矩陣進行訊息傳播，強化節點間的因果一致性，輸出拓模調整後的 Node Measures。

Regime：Regime Reader 從拓模調整後的 Node Measures 讀出當前 Regime，即系統所處的離散認知狀態域。Regime 是認知表徵層傳遞給決策幾何層的核心語意摘要。

4.3 決策幾何層（Regime → Control）

Memory：Memory System 根據當前 Regime 與 Node Measures，從 Episode / Pattern / Impression Tree 中檢索相關歷史記憶，提供 Memory Weight 回饋至 Evidence Fusion（形成閉環），同時提供歷史軌跡參考給 Future Cone。

Future Cone：以當前 Node Measures 與 Regime 為起點，World Model 向前展開 N 步，生成多條候選軌跡，構成 Future Cone。Memory 提供的歷史軌跡可作為 Future Cone 的先驗偏置，提升軌跡採樣效率。

VET Terrain：Memory 與 Future Cone 的輸出在此匯合，VET Functional 對每條候選軌跡計算 V（Value）、E（Entropy/Risk）、T（Terrain）的綜合評分，選出最優軌跡。VET Terrain 是整個決策過程的核心評估器。

Controller → ArduPilot：最優軌跡由 MPC / Policy Adapter 轉換為具體的低層控制命令（推力、姿態、速度指令），以 MAVLink 協議發送至 ArduPilot 執行。

4.4 關鍵設計特性

雙路輸入並行處理 Camera 與 ArduPilot 兩條輸入路徑完全並行，互不依賴，任一路徑的延遲或失效不會阻塞另一路徑。	Memory 的雙向閉環 Memory System 同時向 Evidence Fusion 提供 Memory Weight（影響當前認知狀態）並向 Future Cone 提供歷史軌跡（影響未來規劃），形成認知與規劃的雙向閉環。	單一決策匯聚點 所有資訊流最終匯聚於 VET Terrain，確保決策的一致性與可追溯性。系統不存在多個並行決策路徑，避免決策衝突。	輸出唯一性 Controller 是系統唯一的控制命令輸出點，所有規劃結果必須經過此模組轉換，確保控制命令的格式一致性與安全邊界檢查。
---	--	--	---

Chapter 5. Module Responsibilities

整個系統共有十三個主要模組。

Module	Responsibility
External Encoder	建立外部世界表示
Internal Encoder	建立自身狀態表示
Projection Head	建立固定認知座標
Proxy Generator	建立 Self-supervised Teacher
Evidence Fusion	融合認知證據
Node Topology	五行因果流
Regime Reader	判斷目前 Regime
Memory	長期經驗
Future Cone	多軌跡推演
Terrain Functional	VET 評分
Planner	選最佳 Trajectory
Controller	控制 UAV
Trainer	Online / Offline Learning

以下逐一說明十三個模組的詳細職責、輸入輸出規格、與相鄰模組的介面約束，以及 Codex 實作時的關鍵要求。每個模組對應 Vol. 00 定義的固定格式規範，並在後續各冊中有完整的工程規格展開。

1. External Encoder

職責：將相機影像序列壓縮為外部世界的潛空間表徵 z_{ext} ，捕捉環境的時序動態結構。

- 輸入：**Camera frames，shape: (T, H, W, 3)，float32，30fps
- 輸出：** z_{ext} ，shape: (D_ext,)，float32，代表外部環境的動態壓縮表徵
- 核心模型：**V-JEPA (Video Joint Embedding Predictive Architecture)，以 masked latent prediction 進行自監督訓練，不需要像素重建
- Codex 要求：**實作 VJEPAEncoder 類別，支援 batch inference 與 latent cache

2. Internal Encoder

職責：將 ArduPilot 飛控狀態序列壓縮為無人機自身狀態的潛空間表徵 z_{int} ，並維護對自身未來狀態的預測能力。

- 輸入：**ArduPilot 狀態向量序列，包含速度、姿態、電池、馬達、GPS 等，shape: (T, D_state)
- 輸出：** $z_{int} = (h_t, z_t)$ ，其中 h_t 為確定性隱狀態， z_t 為隨機潛狀態
- 核心模型：**RSSM (Recurrent State Space Model)，同時維護確定性與隨機兩條路徑
- Codex 要求：**實作 InternalRSSM 類別，支援 prior / posterior 分離推理，輸出 KL divergence 供訓練使用

3. Projection Head

職責：將融合後的聯合潛空間 (z_{ext} , z_{int}) 投影至 10 個 Directional Node Embeddings，建立具語意的認知座標系。

- 輸入：**(z_{ext} , z_{int}) 的 concatenation 或 cross-attention 融合結果，shape: (D_fused,)
- 輸出：**10 個 Node Embeddings，shape: (10, D_node)，對應 R+, R-, V+, V-, O+, O-, C+, C-, B+, B-
- 核心模型：**多層 MLP，每個 Node 有獨立的投影頭，以 Proxy Signal 進行語意錨定訓練
- Codex 要求：**實作 ProjectionHead 類別，支援 per-node loss 計算，輸出 node_embeddings dict

4. Proxy Generator

職責：為十個 Directional Nodes 生成自監督教師訊號 (Proxy Signals)，作為 Projection Head 的語意錨定目標。

- 輸入：**Raw sensor data、飛控日誌、World Model rollout 結果
- 輸出：**10 個 proxy scalar values，每個對應一個 Node 的語意目標值
- Proxy 定義：**R±: viability margin 變化率；V±: 目標距離變化；O±: 可行軌跡數量變化；C±: World Model prediction error 倒數；B±: policy consistency score
- Codex 要求：**實作 ProxySignalGenerator 類別，每個 proxy 有獨立的計算函數，支援離線批次計算與在線即時計算兩種模式

5. Evidence Fusion

職責：融合四種 Evidence 來源，得到最終的 10 個 Node Measures，反映多源不確定性的整合認知狀態。

- 輸入：**Node Embeddings (10, D_node)、Probability Evidence、Rollout Weight、Memory Weight、World Model Belief
- 輸出：**10 個 Node Measures，shape: (10,)，每個值域為 [-1, 1]，經過 tanh 正規化
- 核心模型：**Evidence MLP，以四種 Evidence 作為調製訊號，對 Node Embeddings 進行加權融合
- Codex 要求：**實作 EvidenceFusion 類別，支援 evidence_weights 的動態更新，輸出 node_measures tensor

6. Node Topology

職責：依照預定義的因果流向矩陣，對 Node Measures 進行訊息傳播，強化節點間的因果一致性。

- 輸入：**10 個 Node Measures，shape: (10,)
- 輸出：**拓樸調整後的 10 個 Node Measures，shape: (10,)
- 核心結構：**固定的 10×10 edge mask 矩陣，定義允許的因果流向；edge strength 為可學習參數，但 edge mask 不可學習 (固定)
- Codex 要求：**實作 FixedNodeTopology 類別，edge_mask 以 register_buffer 儲存 (不參與梯度更新)，edge_strength 以 nn.Parameter 儲存

7. Regime Reader

職責：從拓樸調整後的 Node Measures 讀出當前 Regime，即系統所處的離散認知狀態域。

- 輸入：**拓樸調整後的 Node Measures，shape: (10,)
- 輸出：**Regime token (離散整數 ID)、Regime embedding (連續向量)、transition probability (轉換至其他 Regime 的機率分布)
- 核心結構：**Regime Codebook (可學習的 K 個 Regime 原型向量) + 最近鄰查找 + Gumbel-Softmax 離散化
- Codex 要求：**實作 RegimeReader 類別，支援 codebook 的離線初始化與在線更新，輸出 regime_id, regime_embedding, transition_logits

8. Memory

職責：維護 VET-C 的長期記憶系統，儲存歷史 Episode、Pattern 與 Impression Tree，並根據當前 Regime 進行相關記憶的檢索。

- 輸入：**當前 Regime、Node Measures、執行結果 (用於寫入)
- 輸出：**Memory Weight (回饋至 Evidence Fusion)、相關歷史軌跡 (提供給 Future Cone)
- 記憶層次：**Log (原始記錄) → Episode (情節摘要) → Pattern (跨情節模式) → Impression Tree (壓縮長期記憶)
- Codex 要求：**實作 MemorySystem 類別，支援 write / retrieve / compress 三種操作，以 SQLite 或 HDF5 作為持久化後端

9. Future Cone

職責：以當前 Node Measures 與 Regime 為起點，使用 World Model 向前展開多條候選軌跡，構成 Future Cone。

- 輸入：**當前 Node Measures、Regime、Memory 提供的歷史軌跡先驗
- 輸出：**N 條候選軌跡，每條軌跡為 T 步的 Node Measures 序列，shape: (N, T, 10)
- 採樣策略：**以 Regime 為條件的 trajectory sampler，結合 Memory 先驗進行重要性採樣
- Codex 要求：**實作 FutureConeEngine 類別，支援 batch rollout，輸出 trajectories tensor 與 feasibility mask

10. Terrain Functional

職責：對 Future Cone 中的每條候選軌跡計算 VET 綜合評分，選出最優軌跡。

- 輸入：**N 條候選軌跡，shape: (N, T, 10)；當前 Regime；任務目標向量
- 輸出：**每條軌跡的 VET score，shape: (N,)；最優軌跡索引
- 評分函數：** $score = \alpha \cdot V_score - \beta \cdot E_score - \gamma \cdot T_score$ ，其中 α, β, γ 為 Regime-dependent 權重
- Codex 要求：**實作 VETFunctional 類別，支援 regime-conditioned 權重查表，輸出 scores tensor 與 best_trajectory_idx

11. Planner

職責：接收 Terrain Functional 選出的最優軌跡，進行最終的規劃決策，輸出目標狀態序列供 Controller 執行。

- 輸入：**最優軌跡 (Node Measures 序列)、當前 Regime、安全邊界約束
- 輸出：**目標狀態序列 (位置、速度、姿態的期望值序列)
- 安全約束：**Planner 必須在輸出前檢查目標狀態序列是否違反 Regime 定義的安全邊界，違反時觸發 fallback 策略
- Codex 要求：**實作 Planner 類別，包含 safety_check 方法，違反安全邊界時返回 safe_fallback_trajectory

12. Controller

職責：將 Planner 輸出的目標狀態序列轉換為具體的低層控制命令，以 MAVLink 協議發送至 ArduPilot 執行。

- 輸入：**目標狀態序列 (位置、速度、姿態期望值)
- 輸出：**MAVLink 控制命令 (SET_POSITION_TARGET_LOCAL_NED 或 SET_ATTITUDE_TARGET)
- 實作選項：**MPC (Model Predictive Control) 或 Policy Adapter (預訓練的低層控制 policy)
- Codex 要求：**實作 Controller 類別，支援 MPC 與 Policy 兩種後端的切換，以 pymavlink 作為 ArduPilot 通訊介面

13. Trainer

職責：管理 VET-C 的完整訓練流程，包含離線預訓練、自監督 rollout 訓練、拓樸訓練與 Test-Time Training (TTT)。

- 訓練階段：**Phase 1: External / Internal Encoder 預訓練；Phase 2: Projection Head + Proxy Signal 訓練；Phase 3: Evidence Fusion + Topology 訓練；Phase 4: End-to-end fine-tuning；Phase 5: TTT (飛行中在線適應)
- TTT 安全約束：**TTT 只允許更新 Projection Head、Evidence Fusion、Memory Weight，不得修改 Regime Codebook 與 Node Topology 的 edge mask
- Codex 要求：**實作 TrainingPipeline 類別，以 loss_registry dict 管理所有 loss 函數，支援 phase-based 訓練排程與 checkpoint 管理

Chapter 6. Core Data Objects

系統中共有八種核心物件。

物件名稱	Python 類別	主要欄位	所屬層
Observation	Observation	camera_frames, imu, gps, battery, motor, ardupilot_state, timestamp	Layer 1
ExternalLatent	ExternalLatent	z_ext: Tensor(D_ext,), timestamp, encoder_version	Layer 2
InternalLatent	InternalLatent	h_t: Tensor(D_h,), z_t: Tensor(D_z,), timestamp	Layer 2
NodeEmbedding	NodeEmbedding	embeddings: Tensor(10, D_node,), node_names: List[str]	Layer 2
NodeMeasure	NodeMeasure	measures: Tensor(10,), regime_id: int, timestamp	Layer 2
Regime	Regime	regime_id: int, regime_embedding: Tensor(D_r,), transition_logits: Tensor(K,)	Layer 2
Trajectory	Trajectory	node_sequences: Tensor(N, T, 10), vet_scores: Tensor(N,), best_idx: int	Layer 3
ControlCommand	ControlCommand	target_position, target_velocity, target_attitude, mavlink_msg_type	Layer 3

⊗ **跨層存取禁止**：Controller 不得直接讀取 Observation 或 Latent。所有資料必須沿 Observation → Latent → Node → Measure → Regime → Trajectory → ControlCommand 的單向鏈路傳遞，不得跨層呼叫。

6.1 Interface 規範

每個資料物件均以 Python `dataclass` 或 `Pydantic BaseModel` 定義，並附帶 `validate()` 方法。模組間的資料傳遞必須使用上述物件，不得以 raw tensor 或 dict 直接跨模組傳遞。所有物件均包含 `timestamp` 欄位，用於時間戳對齊與除錯追溯。

Chapter 8. Design Principles

VET-C 有七條核心原則。

Principle 1 : Representation $\neq$ Cognition

Latent 是壓縮表徵，不是理解。唯有經過 Projection Head 投影至 Node Space 之後，系統才開始進行認知推理。

Principle 2 : Node $\neq$ Regime

Node 代表單一認知維度的局部方向強度（如 R+ 或 V-）。Regime 是所有 Node 的整體組合狀態，代表系統當前所處的認知域。

Principle 3 : Evidence $\neq$ Measure

Evidence 是來自不同來源的原始信號（Probability、Rollout Weight、Memory Weight、World Model Belief）。Node Measure 才是融合後的最終 Node Activation，代表系統的真實認知強度。

Principle 4 : Memory $\neq$ Belief

Memory 儲存的是過去發生的事實記錄（Log、Episode、Pattern）。Belief 代表系統在當前時刻仍然認為成立的信念，是 Memory 經過 Evidence Fusion 後的現時推論結果。

Principle 5 : Topology is Fixed

十節點的因果拓樸結構（edge mask）是固定的，不可學習，不可在線修改。只有 Edge Strength（邊的強度）是可學習參數，確保因果結構的穩定性與安全性。

Principle 6 : Projection is Learnable, Topology is Not

Projection Head 是可學習的——它負責將 latent 映射至語意 Node Space，需要透過 Proxy Signal 訓練。Node Topology 的 edge mask 不可學習，是系統安全邊界的架構級保證。

Principle 7 : Planning is Functional Optimization

最終決策不是由 Policy Network 直接輸出控制命令，而是在 Future Cone 中展開多條候選軌跡，由 VET Functional 進行最佳化評分，選出最優路徑後再轉換為控制命令。

Chapter 9. Repository Layout

目錄路徑	對應模組	主要內容
vetc/configs/	全域設定	超參數、模型設定、訓練設定的 YAML 配置檔
vetc/datasets/	資料集管線	感測器資料載入、時間對齊、資料集分割
vetc/encoders/external/	External Encoder	V-JEPA wrapper、video encoder、latent cache
vetc/encoders/internal/	Internal Encoder	RSSM model、state encoder、prior/posterior
vetc/projection/	Projection Head	ProjectionHead、per-node MLP、proxy grounding loss
vetc/proxy/	Proxy Generator	ProxySignalGenerator、每個 proxy 的計算函數
vetc/evidence/	Evidence Fusion	EvidenceFusion MLP、evidence weight scheduler
vetc/topology/	Node Topology	FixedNodeTopology、edge mask、edge strength
vetc/regime/	Regime Reader	RegimeReader、codebook、Gumbel-Softmax
vetc/memory/	Memory System	MemorySystem、write/retrieve/compress、SQLite/HDF5 後端
vetc/rollout/	Future Cone	FutureConeEngine、trajectory sampler、feasibility mask
vetc/terrain/	Terrain Functional	VETFunctional、VET scorer、trajectory ranking
vetc/planner/	Planner	Planner、safety_check、fallback strategy
vetc/controller/	Controller	Controller、MPC/Policy adapter、pymavlink 介面
vetc/training/	Trainer	TrainingPipeline、loss_registry、phase scheduler、checkpoint
vetc/tests/	測試套件	各模組 unit tests、integration tests、simulation tests
vetc/schemas/	資料物件定義	Observation、Latent、NodeMeasure、Regime、Trajectory、ControlCommand

❏ 每個子目錄均包含 `__init__.py`、`model.py`（模型定義）、`interface.py`（輸入輸出介面）、`train.py`（訓練邏輯，若適用）與 `test_*.py`（單元測試）。模組間依賴關係嚴格遵循 Chapter 6 定義的資料物件介面，不得直接跨目錄存取內部實作。

Chapter 10. Engineering Roadmap

整個工程依照下面順序完成。



Phase 1 — Encoder & Latent Space

建立 External Encoder (V-JEPA) 與 Internal Encoder (RSSM)，驗證 z_{ext} 與 z_{int} 的輸出格式與 latent cache 機制。



Phase 4 — Node Topology & Regime

建立 Fixed Node Topology (固定 edge mask + 可學習 edge strength) 與 Regime Reader (Codebook + Gumbel-Softmax)，驗證 Regime 離散化的穩定性。



Phase 7 — Planner & Controller

建立 Planner (含 safety_check) 與 Controller (MPC/Policy adapter + pymavlink 介面)，驗證端對端控制命令輸出。



Phase 2 — Projection & Proxy Signals

建立 Projection Head 與 Proxy Signal Generator，以 proxy grounding loss 訓練 10 個 Node Embeddings 的語意錨定。



Phase 5 — Memory System

建立 Memory System (Log → Episode → Pattern → Impression Tree)，驗證 write / retrieve / compress 三種操作與持久化後端。



Phase 8 — Training Pipeline

建立完整 Training Pipeline (Phase 1–4 訓練排程 + loss_registry + checkpoint 管理)，執行離線預訓練與端對端 fine-tuning。



Phase 3 — Evidence Fusion & Node Measures

建立 Evidence Fusion MLP，融合四種 Evidence 得到 10 個 Node Measures，驗證 tanh 正規化與 evidence weight 動態更新。



Phase 6 — Future Cone & VET Terrain

建立 Future Cone Engine (World Model rollout + trajectory sampling) 與 VET Functional (V/E/T 評分)，驗證軌跡排序的正確性。



Phase 9 — Test-Time Training & Deployment

建立 TTT 在線適應機制 (只更新 Projection Head / Evidence Fusion / Memory Weight)，完成模擬測試與實機部署驗收。

Chapter 11. Acceptance Criteria

完成 Vol.00 時，Codex 應能：

- 建立完整 Repository。
- 建立所有 Module Interface（空殼即可）。
- 定義核心資料結構（Observation、Latent、Node、Measure、Regime、Memory、Trajectory、Control）。
- 建立模組依賴關係與資料流，不允許跨層直接存取。
- 產生主程式流程與配置檔，讓後續各 Volume 可以逐步填入實作。

End of Volume 1

後續 Volume 的預告:

下一冊：Volume 01

下一冊 **Vol.01 – Data Interface & Sensor Pipeline** 將開始定義所有資料格式（Camera、ArduPilot、IMU、GPS、Battery、Motor、Mission、Log）、時間同步、ROS2 Topic（若使用）、以及 Observation Schema。這一冊完成後，Codex 就可以開始建立真正的資料流與資料集管線。